Course Code: INT 315

Course Title: Cluster Computing

Continuous Assessment: 02

Date Of Submission: 09-April-2026

Topic: Renewable Energy Output Prediction

Name: Aditya Verma

Registration No.: 12216333

Roll No.: REM301A62

Section: EM301

TABLE OF CONTENT

TITLE 1

ABSTRACT 1

INTRODUCTION 1

PROBLEM STATEMENT 1

OBJECTIVES 2

DATASET DESCRIPTION 2

DATA PREPROCESSING 2

FEATURE ENGINEERING 3

MODEL BUILDING 3

EVALUATION METRICS 3

RESULTS 4

VISUALIZATION 4

DISCUSSION 5

CONCLUSION 5

FUTURE WORK 5

IMPLEMENTATION 6

LINKS 11

TITLE

Renewable Energy Output Prediction using Linear Regression (PySpark)

ABSTRACT

Renewable energy forecasting is essential for efficient power management and grid stability. This project focuses on predicting power output using environmental parameters such as temperature, humidity, dew point, and wind speed. A Linear Regression model is implemented using PySpark to handle large-scale data efficiently. The dataset is preprocessed to remove missing values and normalize features. The model is evaluated using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). Visualization techniques such as scatter plots, heatmaps, and error distributions are used to analyze model performance. The results indicate moderate accuracy, highlighting the limitations of linear models for inherently non-linear energy data.

INTRODUCTION

Renewable energy sources like wind and solar play a critical role in reducing carbon emissions and ensuring sustainable development. However, these energy sources are highly dependent on environmental conditions, making their output unpredictable. Accurate prediction of energy output is essential for efficient energy distribution and grid management.

Machine learning techniques have become popular for forecasting renewable energy output. Among them, Linear Regression is one of the simplest and most interpretable models. This project explores the use of Linear Regression to predict energy output using environmental variables.

PROBLEM STATEMENT

The objective of this project is to predict renewable energy output based on environmental factors such as temperature, humidity, and wind speed. The challenge lies in modeling the relationship between these variables and power generation, especially when the relationship is not strictly linear.

OBJECTIVES

- To load and preprocess renewable energy dataset using PySpark
- To handle missing values effectively
- To select relevant environmental features
- To build a Linear Regression model
- To evaluate the model using RMSE and MAE
- To visualize results using graphs

DATASET DESCRIPTION

The dataset contains environmental parameters and corresponding power output. The key features include:

- Temperature at 2 meters
- Relative humidity
- Dew point
- Wind speed at 10m and 100m
- Wind gusts
- Power output (target variable)

The dataset consists of thousands of observations recorded over time. Missing values were removed to ensure data quality.

DATA PREPROCESSING

Data preprocessing is a crucial step in machine learning. The following steps were performed:

1. Handling Missing Values
   Rows with missing values were removed using dropna().
2. Feature Selection
   Relevant features influencing power generation were selected.

3. Feature Transformation
   Features were combined into a vector using VectorAssembler.
4. Feature Scaling
   StandardScaler was applied to normalize the features, ensuring that all variables contribute equally to the model.

FEATURE ENGINEERING

Feature engineering involves selecting meaningful input variables. The selected features in this project include:

- temperature_2m
- relativehumidity_2m
- dewpoint_2m
- windspeed_10m
- windspeed_100m
- windgusts_10m

These features are known to significantly affect renewable energy generation, especially wind-based power.

MODEL BUILDING

A Linear Regression model was implemented using PySpark MLlib. The dataset was split into training and testing sets in an 80:20 ratio.

Regularization was applied to prevent overfitting and improve model stability. The model learns the relationship between environmental features and power output.

EVALUATION METRICS

Two metrics were used:

- RMSE (Root Mean Squared Error)
  Measures the overall magnitude of prediction errors.
- MAE (Mean Absolute Error)
  Measures the average absolute difference between predicted and actual values.

These metrics provide insight into model accuracy.

RESULTS

The model achieved:

- RMSE ≈ 0.18
- MAE ≈ 0.14

These values indicate moderate prediction accuracy. The model captures general trends but struggles with precise predictions.

VISUALIZATION

Scatter Plot

The scatter plot shows the relationship between actual and predicted values. Ideally, points should lie on a straight diagonal line. The spread indicates prediction error.

Heatmap

The heatmap shows correlation between features. Wind speed features show strong positive correlation with power output, indicating their importance.

Error Distribution

The error distribution graph shows that most errors are centered around zero, indicating unbiased predictions. However, some spread exists, showing limitations in accuracy.

DISCUSSION

The Linear Regression model performs moderately well but has limitations. Renewable energy output depends on complex, non-linear relationships between environmental variables. Linear models cannot fully capture these interactions.

The heatmap confirms that wind speed plays a major role in power generation. However, other variables interact in non-linear ways, which reduces model accuracy.

CONCLUSION

This project successfully implemented a Linear Regression model using PySpark for renewable energy prediction. The model achieved moderate accuracy and demonstrated the importance of feature selection and preprocessing.

Although Linear Regression is simple and interpretable, it is not sufficient for capturing complex patterns in energy data.

FUTURE WORK

Future improvements include:

- Using non-linear models such as Random Forest and Gradient Boosting
- Applying feature engineering techniques
- Incorporating time-series analysis
- Using deep learning models for better accuracy

IMPLEMENTATION

Step 1: Load Libraries

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> pyspark
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
26/04/08 17:58:20 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException
: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/08 17:58:23 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
 classes where applicable
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 4.1.1
      /_/

Using Python version 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023 05:45:37)
Spark context Web UI available at http://Invalid.bbrouter:4040
Spark context available as 'sc' (master = local[*], app id = local-1775651307605).
SparkSession available as 'spark'.
>>> from pyspark.sql import SparkSession
```

Output: Enables Spark environment to process large datasets.

Step 2: Create Spark Session

```
>>> spark = SparkSession.builder.appName("RenewableEnergyPrediction").getOrCreate()
26/04/08 17:59:48 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
>>>
```

Output: Initializes Spark application for data processing.

Step 3: Load Dataset

```
>>> df = spark.read.csv("C:/Users/Aditya/Projects/Renewable Energy Output Prediction/Location1.csv", header=True, inferSchema=True)
```

Output: Dataset loaded into Spark DataFrame with proper column types.

Step 4: Handle Missing Values

```
>>> df = df.dropna()
>>>
```

Output: Removes rows with null values to ensure clean data.

Step 5: Define Features and Target

```
>>> features = ["temperature_2m","relativehumidity_2m","dewpoint_2m","windspeed_10m","windspeed_100m","windgusts_10m"]
>>> target = "Power"
>>>
```

Output: Selects input variables and prediction target.

Step 6: Feature Assembly

```
>>> from pyspark.ml.feature import VectorAssembler
>>> assembler = VectorAssembler(inputCols=features, outputCol="unscaled")
>>> data = assembler.transform(df)
>>>
```

Output: Combines multiple columns into a single feature vector.

Step 7: Feature Scaling

```
>>> from pyspark.ml.feature import StandardScaler
>>> scaler = StandardScaler(inputCol="unscaled", outputCol="features")
>>> scaled_data = scaler.fit(data).transform(data)
>>>
```

Output: Normalizes features so all variables contribute equally.

Step 8: Prepare Final Data

```
>>>
>>> from pyspark.sql.functions import col
>>> final_data = scaled_data.select("features", col(target).alias("label"))
>>>
```

Output: Creates dataset with features and label for model training.

Step 9: Train-Test Split

```
>>>
>>> train_data, test_data = final_data.randomSplit([0.8, 0.2], seed=42)
>>>
```

Output: Splits data into training (80%) and testing (20%).

Step 10: Train Model

```
>>> from pyspark.ml.regression import LinearRegression
>>> lr = LinearRegression(featuresCol="features", labelCol="label", regParam=0.1)
>>> model = lr.fit(train_data)
26/04/08 18:00:04 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
26/04/08 18:00:05 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK
>>>
```

Output: Trains Linear Regression model on training data.

Step 11: Make Predictions

```
>>>
>>> predictions = model.transform(test_data)
>>>
```

Output: Generates predicted power values for test data.

Step 12: Evaluate Model

```
>>> from pyspark.ml.evaluation import RegressionEvaluator
>>> rmse = RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse").evaluate(predictions)
>>> mae = RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="mae").evaluate(predictions)
>>>
>>> print("RMSE:", rmse)
RMSE: 0.1838971758875166
>>> print("MAE:", mae)
MAE: 0.14831533998051866
>>>
```

Output:

- RMSE ≈ 0.18 → average prediction error magnitude
- MAE ≈ 0.14 → average absolute error

Step 13: Convert to Pandas

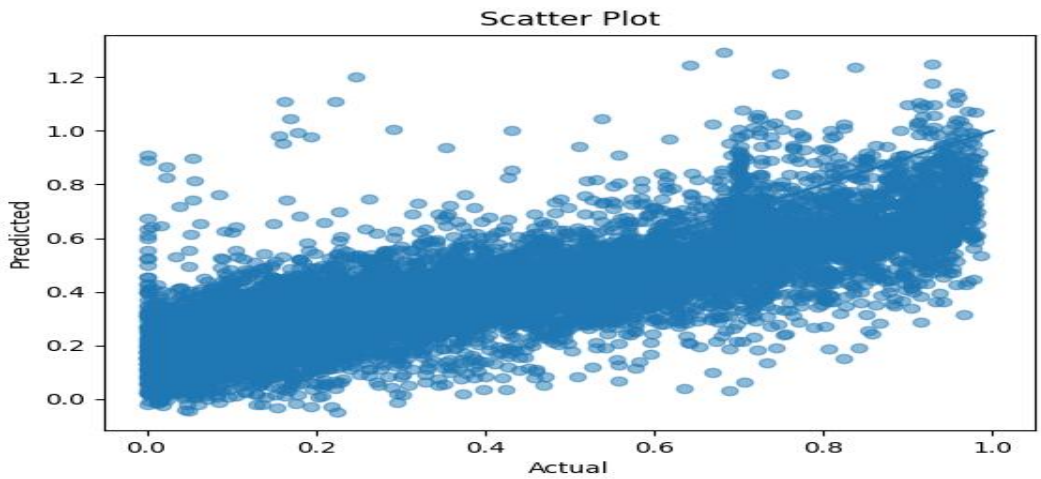
```
>>> pandas_df = predictions.select("label","prediction").toPandas()
>>>
```

Output: Converts Spark DataFrame to Pandas for visualization.

Step 14: Scatter Plot

```
>>> import matplotlib.pyplot as plt
>>> plt.figure();
<Figure size 640x480 with 0 Axes>
>>> plt.scatter(pandas_df["label"], pandas_df["prediction"], alpha=0.5);
<matplotlib.collections.PathCollection object at 0x000002D4D02547D0>
>>> plt.plot([0,1],[0,1]);
[<matplotlib.lines.Line2D object at 0x000002D4CF4D6950>]
>>> plt.xlabel("Actual");
Text(0.5, 0, 'Actual')
>>> plt.ylabel("Predicted");
Text(0, 0.5, 'Predicted')
>>> plt.title("Scatter Plot");
Text(0.5, 1.0, 'Scatter Plot')
>>> plt.savefig("C:/Users/Aditya/Desktop/scatter.png");
>>> plt.close();
```

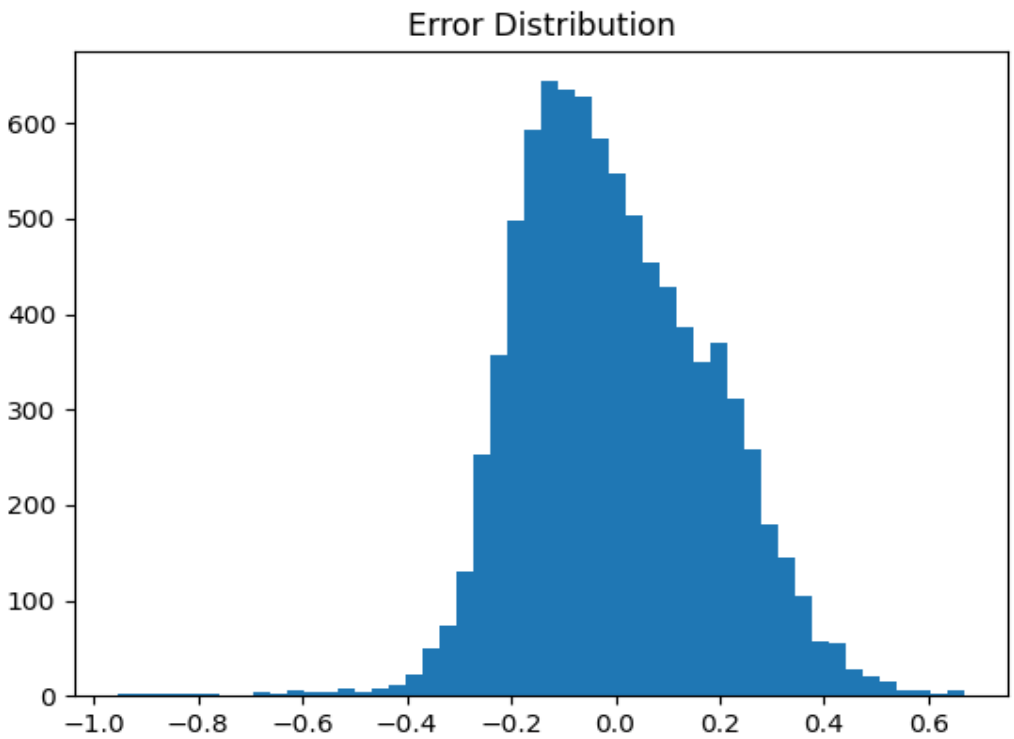
Output: Shows relationship between actual and predicted values.



Step 15: Error Distribution

```
>>> pandas_df["error"] = pandas_df["label"] - pandas_df["prediction"]
>>> plt.figure();
<Figure size 640x480 with 0 Axes>
>>> plt.hist(pandas_df["error"], bins=50);
(array([  2.,   2.,   2.,   2.,   3.,   2.,   1.,   1.,   4.,   2.,   5.,
         4.,   4.,   7.,   4.,   7.,  12.,  22.,  50.,  73., 130., 252.,
       357., 498., 594., 644., 636., 628., 584., 547., 504., 455., 429.,
       386., 350., 370., 311., 258., 179., 144., 105.,  57.,  55.,  28.,
        20.,  15.,   6.,   5.,   3.,   5.]), array([-0.95522879, -0.9227184 , -0.890208  , -0.8576976 , -0.8251872 ,
       -0.79267681, -0.76016641, -0.72765601, -0.69514561, -0.66263522,
       -0.63012482, -0.59761442, -0.56510402, -0.53259363, -0.50008323,
       -0.46757283, -0.43506244, -0.40255204, -0.37004164, -0.33753124,
       -0.30502085, -0.27251045, -0.24000005, -0.20748965, -0.17497926,
       -0.14246886, -0.10995846, -0.07744806, -0.04493767, -0.01242727,
        0.02008313,  0.05259353,  0.08510392,  0.11761432,  0.15012472,
        0.18263512,  0.21514551,  0.24765591,  0.28016631,  0.31267671,
        0.3451871 ,  0.3776975 ,  0.4102079 ,  0.4427183 ,  0.47522869,
        0.50773909,  0.54024949,  0.57275989,  0.60527028,  0.63778068,
        0.67029108]), <BarContainer object of 50 artists>)
>>> plt.title("Error Distribution");
Text(0.5, 1.0, 'Error Distribution')
>>> plt.savefig("C:/Users/Aditya/Desktop/error.png");
>>> plt.close();
```

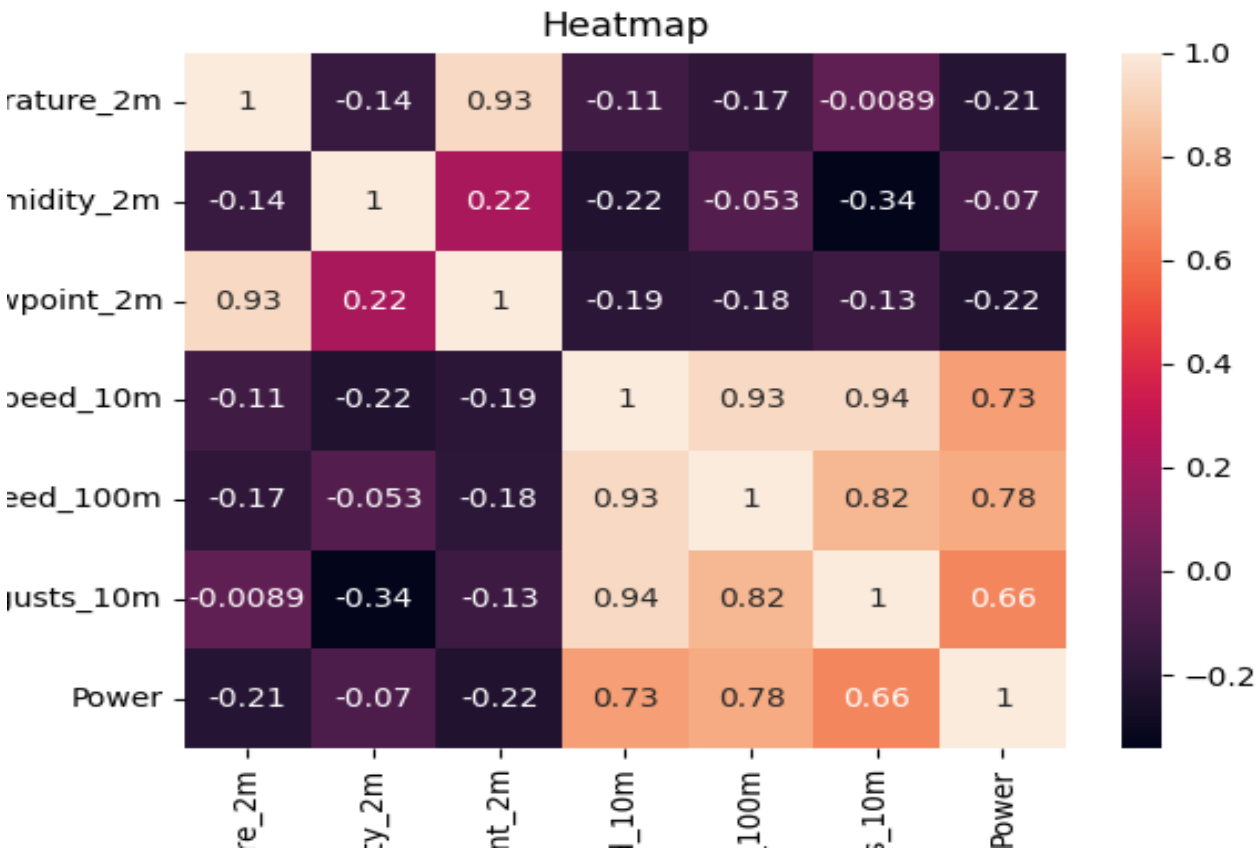
Output: Displays distribution of prediction errors (centered near zero).



Step 16: Heatmap

```
>>> heat_df = df.select("temperature_2m","relativehumidity_2m","dewpoint_2m","windspeed_10m","windspeed_100m","windgusts_10m","Power").toPandas()
>>>
>>> import seaborn as sns
>>> plt.figure();
<Figure size 640x480 with 0 Axes>
>>> sns.heatmap(heat_df.corr(), annot=True);
<Axes: >
>>> plt.title("Heatmap");
Text(0.5, 1.0, 'Heatmap')
>>> plt.savefig("C:/Users/Aditya/Desktop/heatmap.png");
>>> plt.close();
>>> 
```

Output: Shows correlation between features and power output.



LINKS

GITHUB LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/tree/main

WORD LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Report.docx

PDF LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Report.pdf

PPT LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Renewable-Energy-Output-Prediction.pptx